

Requirement Analysis & Customer Database Module

System Design Document

Version 1.0

1. Introduction

This document defines the requirements and database design for the **Customer Database Module**, a core component responsible for storing, organizing, and maintaining customer information for the system. It covers the purpose, scope, functional and non-functional requirements, and the underlying relational database schema, including entity relationships and table definitions.

1.1 Purpose

The purpose of this module is to provide a centralized, reliable, and secure store for customer profile data, contact details, addresses, segmentation, interaction history, and audit trails, enabling other modules (sales, support, marketing, billing) to consume accurate and up-to-date customer information.

1.2 Scope

- Create, read, update, and soft-delete customer profiles.
- Manage multiple contact methods (email, phone) per customer.
- Manage multiple addresses per customer with a designated default.
- Group customers into segments for marketing/sales targeting.
- Log customer interactions and support notes.
- Maintain an audit trail of changes to customer records.

2. Functional Requirements

ID	Requirement	Priority
FR-01	System shall allow creation of a new customer profile with mandatory first name, last name, and status.	High
FR-02	System shall allow a customer to have multiple contact methods, with exactly one primary email and one primary phone.	High
FR-03	System shall validate email format before saving a contact method.	High
FR-04	System shall allow a customer to have multiple addresses, with exactly one marked as default.	High
FR-05	System shall allow customers to be assigned to one or more segments (e.g. VIP, Wholesale, Newsletter).	Medium
FR-06	System shall allow staff to log interaction notes against a customer record (calls, emails, chats).	Medium
FR-07	System shall record an audit entry whenever a customer's profile field is changed.	High
FR-08	System shall support searching/filtering customers by name, email, phone, or segment.	Medium
FR-09	System shall support soft status transitions (ACTIVE, INACTIVE, SUSPENDED, DELETED) rather than hard deletes.	Medium

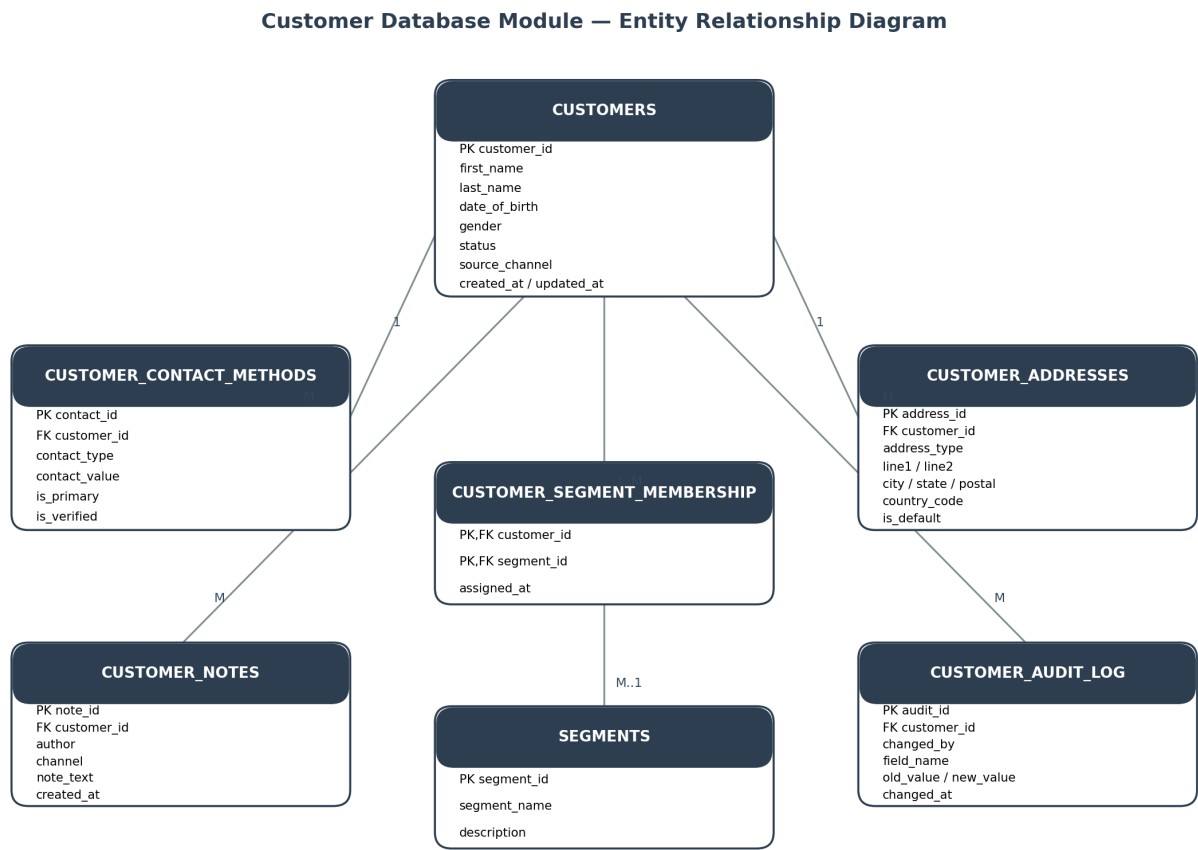
3. Non-Functional Requirements

Category	Requirement
Performance	Customer lookup by email/phone shall return results in under 200ms for a database of up to 5 million records.
Security	All personally identifiable information (PII) shall be encrypted at rest and in transit; access shall be role-based.
Data Integrity	Referential integrity shall be enforced via foreign keys; no orphaned contact/address/note records shall exist.
Availability	The module shall support 99.9% uptime for read operations.
Scalability	Schema shall support horizontal partitioning by customer_id if record volume exceeds 20 million rows.
Auditability	All profile field changes shall be traceable to a user and timestamp for a minimum of 7 years.
Compliance	Design shall support data subject access and erasure requests (e.g. GDPR Article 17).

4. Database Design

The schema below normalizes customer data into seven related tables. Contact methods and addresses are split into their own one-to-many tables to support multiple entries per customer. Segmentation uses a many-to-many junction table. Notes and audit logs provide interaction history and change tracking respectively.

4.1 Entity Relationship Diagram



4.2 Relationship Summary

Parent	Child	Cardinality
customers	customer_contact_methods	1 : M
customers	customer_addresses	1 : M
customers	customer_notes	1 : M
customers	customer_audit_log	1 : M
customers	customer_segment_membership	1 : M
segments	customer_segment_membership	1 : M

4.3 Table Definitions

customers

Core customer profile record.

Column	Type	Constraints
customer_id	UUID	PK, default random
first_name	VARCHAR(100)	NOT NULL
last_name	VARCHAR(100)	NOT NULL
date_of_birth	DATE	
gender	VARCHAR(20)	
status	VARCHAR(20)	NOT NULL, default ACTIVE, CHECK IN (ACTIVE/INACTIVE/SUSPENDED/DELETED)
source_channel	VARCHAR(50)	e.g. WEB, MOBILE_APP, IN_STORE, REFERRAL
created_at / updated_at	TIMESTAMPTZ	NOT NULL, auto-managed via trigger

customer_contact_methods

Stores one or more email/phone contacts per customer.

Column	Type	Constraints
contact_id	UUID	PK, default random
customer_id	UUID	FK -> customers, ON DELETE CASCADE
contact_type	VARCHAR(10)	CHECK IN (EMAIL, PHONE)
contact_value	VARCHAR(255)	NOT NULL, email format validated
is_primary	BOOLEAN	one primary per type per customer (unique partial index)
is_verified / verified_at	BOOLEAN / TIMESTAMPTZ	

customer_addresses

Stores one or more addresses per customer.

Column	Type	Constraints
address_id	UUID	PK, default random
customer_id	UUID	FK -> customers, ON DELETE CASCADE
address_type	VARCHAR(20)	CHECK IN (HOME/BILLING/SHIPPING/WORK/OTHER)
line1 / line2	VARCHAR(255)	line1 NOT NULL
city / state_province / postal_code	VARCHAR	city NOT NULL
country_code	CHAR(2)	NOT NULL, ISO 3166-1 alpha-2
is_default	BOOLEAN	one default per customer (unique partial index)

segments

Lookup table of customer segments.

Column	Type	Constraints
segment_id	SERIAL	PK
segment_name	VARCHAR(100)	NOT NULL, UNIQUE
description	TEXT	

customer_segment_membership

Many-to-many join between customers and segments.

Column	Type	Constraints
customer_id	UUID	PK, FK -> customers
segment_id	INT	PK, FK -> segments
assigned_at	TIMESTAMPTZ	NOT NULL, default now()

customer_notes

Interaction / support history log.

Column	Type	Constraints
note_id	UUID	PK, default random
customer_id	UUID	FK -> customers, ON DELETE CASCADE
author	VARCHAR(100)	NOT NULL, staff username or system
channel	VARCHAR(30)	e.g. CALL, EMAIL, CHAT, SYSTEM
note_text	TEXT	NOT NULL
created_at	TIMESTAMPTZ	NOT NULL, default now()

customer_audit_log

Field-level change history for compliance.

Column	Type	Constraints
audit_id	BIGSERIAL	PK
customer_id	UUID	FK -> customers, ON DELETE CASCADE
changed_by	VARCHAR(100)	NOT NULL
field_name	VARCHAR(100)	NOT NULL
old_value / new_value	TEXT	
changed_at	TIMESTAMPTZ	NOT NULL, default now()

5. Key Integrity Rules

- Email addresses are unique across the customer base (case-insensitive).
- Only one contact method may be marked primary per type (EMAIL/PHONE) per customer.
- Only one address may be marked default per customer.
- All child tables cascade-delete when a parent customer is removed.
- `updated_at` on `customers` and `customer_addresses` is auto-maintained via a database trigger.

6. Conclusion

This design provides a normalized, extensible foundation for customer data management, supporting multiple contacts and addresses, segmentation, interaction history, and full auditability. It is ready for implementation in PostgreSQL (or adaptable to MySQL/SQL Server) and can be extended with order, billing, or loyalty modules that reference `customers.customer_id` as a foreign key.